

CA2

Name: Alireza Karimi

SID: 810101492

This is the CA2 of the distributed machine learning systems course in University of Tehran.

Some pre-needs

I add the sabalan1 in the /etc/hosts file and i accessed that by:

```
sudo nano /etc/hosts
```

and added this:

```
172.18.32.200 dml0
172.18.32.201 dml1
172.18.32.202 dml2
172.18.32.203 dml3

172.18.32.194 sabalan1
```

and to enter that, i should run this:

```
ssh karimi@sabalan1
```

and also enter my password which is : S1apjFHbHtLq

but i don't want to enter the password each time. so i enter this:

```
ssh-copy-id karimi@sabalan1
```

and i enter my password and from now on, i can use sabalan1 without entering password everytime.

this is how i sync the files from my computer to the sabalan server (run this command before entering the sabalan1 server):

```
rsync -avz --progress /Users/alireza/Documents/uni/term7/DMLS/CA/2/ karimi@sabalan1:~/CA2
```

to exit the server, you simply enter:

```
exit
```

use the `nvidia-smi` command to see the state of gpu and memory information in the sabalan1

and this code is necessary in order to run the question 2 python files in the sabalan1 server:

```
source /home/dmls/pytorch/bin/activate
```

For the first question, in the google colab, change the runtime to use GPU.

Q1

1.1 Basic Concepts (5 Points)

1. What is a CUDA Kernel and what does it do?

Definition:

A CUDA Kernel is a function written in C/C++ (typically decorated with the `__global__` specifier) that acts as the entry point for code execution on the GPU (Device) when called from the CPU (Host)¹¹¹¹.

How it works:

- **SIMT Architecture:** Unlike a standard CPU function that runs sequentially, a kernel follows the **Single Instruction, Multiple Threads (SIMT)** architecture. When a kernel is launched, thousands (or millions) of CUDA threads are spawned.
- **Parallel Execution:** Every single thread executes the **exact same code** (the kernel logic), but they operate on **different portions of data**.
- **Indexing:** Each thread has a unique ID (calculated via `threadIdx`, `blockIdx`, and `blockDim`). In your assignment, the kernel uses these IDs to calculate which specific pixel of the image (or element of the tensor) it is responsible for calculating the Convolution and ReLU for.
- **Role in this Assignment:** The kernel you will write replaces the high-level PyTorch function. Instead of PyTorch iterating over the image or calling a library, your kernel will manually compute the weighted sum of pixels (Convolution) and apply the threshold (ReLU) directly in GPU memory²²²².

2. cuDNN: Definition, Advantages, and Relationship with cuBLAS

What is cuDNN?

cuDNN (CUDA Deep Neural Network library) is a library of primitives specifically optimized for deep neural networks. It provides highly tuned implementations for standard routines such as forward and backward convolution, pooling, normalization, and activation layers³.

Why do PyTorch and other frameworks use it?

- **Performance:** cuDNN is significantly faster than "naive" CUDA implementations. NVIDIA engineers optimize these primitives at the assembly level for specific GPU architectures (e.g., Volta, Ampere, Hopper).

•

Algorithm Selection: For a standard operation like a 3×3 convolution, there are multiple mathematical ways to compute it (GEMM-based, Winograd, FFT, Direct). cuDNN uses heuristics to automatically select the fastest algorithm based on the input tensor size and available memory⁴.

What problem does it solve?

It solves the complexity of hardware optimization.

- Writing a kernel that optimally uses GPU Registers, Shared Memory, and Tensor Cores is extremely difficult and time-consuming.
- cuDNN abstracts this away, allowing frameworks like PyTorch to run efficiently on new hardware without

rewriting the codebase for every new GPU generation.

Relationship between cuDNN and cuBLAS:

- **cuBLAS (CUDA Basic Linear Algebra Subroutines):** This library is dedicated to general linear algebra, primarily **Matrix Multiplication (GEMM)**.
- **The Connection:** Deep learning is fundamentally built on linear algebra.
 - Historically, a convolution could be converted into a matrix multiplication using an operation called `im2col` (image-to-column). In this approach, cuDNN would internally call **cuBLAS** to perform the heavy matrix multiplication.
 - However, modern cuDNN often includes specialized kernels (like the "fused" kernel you are writing in this assignment) that perform convolution *without* strictly converting to matrix multiplication first, or by using specialized algorithms (like Winograd) that are distinct from generic BLAS operations.

Part 1.2: Implementation of Fused Convolution + ReLU Kernel (15 Points)

Your implementation correctly addresses the assignment requirements. Here is the "profound" breakdown of why this code works and how it achieves the "Fused" status.

1. The Python Reference (`conv_relu_py`)

This function serves as the "Gold Standard" or ground truth. It implements the logic explicitly so you can verify the CUDA kernel's accuracy.

- **Logic:** It iterates through every pixel (n, h, w) of the output.
- **The Convolution:** For each output pixel, it looks at the 3×3 neighborhood in the input.
- **Padding Logic:** `in_h = h_idx + kh - 1` correctly implements `padding=1`. When `kh=0`, the index shifts back by 1. The `if` condition handles the zero-padding (boundary check).
- **Reference Accuracy:** The assertions at the start correctly enforce the constraints ($C_{in}=1$, 3×3 Kernel).

2. The CUDA Kernel (`conv_relu_kernel`)

This is the core of the assignment. It maps the Python logic to parallel GPU threads.

Code Explanation & Analysis:

- **Thread Mapping** ($i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$):
Instead of nested loops for N, H, W like the CPU version, you launch a 1D grid of threads. Each thread is responsible for exactly one output pixel.
- **Index Reconstruction:**
Because the threads are 1D but the tensor is 4D (technically 3D here since $C=1$), the code must reverse-engineer the coordinates:
 - `n = i / (H * W)` determines which image in the batch.
 - `rem % W` determines the column (x coordinate).
 - `rem / W` determines the row (y coordinate).
 - *Why this works:* PyTorch tensors are stored in **Row-Major** contiguous memory.
- **The "Fused" Operation:**
 - In a standard PyTorch call (`F.relu(conv(x))`), the GPU would:
 1. Compute Conv results.
 2. Write results to VRAM (Global Memory).
 3. Read results back from VRAM.
 4. Apply ReLU.

5. Write final results to VRAM.

◦ **Your Code:**

1. Computes Conv results in a register (`acc`).
2. Applies ReLU (`fmaxf`) directly on the register.
3. Writes only the final result to VRAM.

◦ **Benefit:** This saves massive amounts of **Memory Bandwidth**, which is usually the bottleneck in deep learning.

• Memory Access (`__restrict__`):

You used `const float* __restrict__ x`. This tells the compiler that the pointer `x` does not overlap with `out`. This allows the NVIDIA compiler (NVCC) to optimize memory loads more aggressively (e.g., caching input values in L1 cache).

Part 1.3: Registering the Kernel in PyTorch (5 Points)

The assignment asks for the `FusedConvReLUFn` class explanation. While your snippet included the C++ compilation (`load_inline`), the actual *PyTorch Layer registration* usually happens in a Python class inheriting from `torch.autograd.Function`.

Since you haven't pasted the `FusedConvReLUFn` class yet, here is the explanation of **what it does and why it is necessary**, which answers the theoretical part of Section 1.3.

Question: Explain the `FusedConvReLUFn` class and its interaction with autograd.

Answer:

1. The Bridge between Python and C++:

PyTorch's autograd system needs to know how to differentiate operations to calculate gradients. A raw C++ function (`conv_relu_fused`) knows how to compute the output, but it doesn't know about gradients. The `FusedConvReLUFn` class bridges this gap.

2. Structure of the Class:

It inherits from `torch.autograd.Function` and defines two static methods:

◦ `forward(ctx, x, w, b)` :

- It receives input tensors.
- It calls your optimized C++ function (`module.conv_relu_fused(x, w, b)`).
- **Crucial Step:** It uses `ctx.save_for_backward(x, w, out)` (or similar) to stash the tensors needed to calculate gradients later.

◦ `backward(ctx, grad_output)` :

- This method is called during `loss.backward()`.
- It retrieves the saved tensors.
- It calculates the gradients with respect to the input (∇x), weights (∇w), and bias (∇b).
- **Note:** Since ReLU is fused, the backward pass must mathematically account for the derivative of ReLU (which is 1 if $x > 0$, else 0) combined with the convolution gradient.

3. Why do we need it?

Without this class, you could only use your kernel for inference (prediction). You could not train the network because PyTorch wouldn't know how to pass the error signal back through your custom kernel.

1.3 Registering the Kernel as a PyTorch Layer (5 Points)

The `FusedConvReLUFn` class serves as the **bridge** between your high-performance C++/CUDA kernel and PyTorch's automatic differentiation engine (`autograd`). Without this class, your kernel could compute results, but the network could not learn from them because PyTorch wouldn't know how to calculate the gradients.

Here is the breakdown of the class methods:

1. Inheritance from `torch.autograd.Function`

Python

```
class FusedConvReLUFn(torch.autograd.Function):
```

By inheriting from this class, you are defining a custom operation with its own forward and backward logic. This bypasses the standard PyTorch computation graph for this specific operation, allowing you to inject your custom CUDA code.

2. The `forward` Method (The Forward Pass)

This method defines how to compute the output from the inputs.

- **ctx (Context):** The first argument is a context object. It acts as a storage container used to pass information from the forward pass to the backward pass.

- **Executing the Kernel:**Python

```
y = module.conv_relu_fused(x, weight, bias)
```

This calls the C++ CUDA extension you compiled earlier. It performs the convolution and ReLU in a single fused step on the GPU.

- **Saving for Backward:**Python

```
ctx.save_for_backward(x, weight, bias, y)
```

This is crucial for Autograd. To calculate gradients later (using the Chain Rule), we often need the original input `x` or the output `y`. Since GPU memory is freed after operations, we must explicitly tell PyTorch to "stash" these tensors in memory until the backward pass is called.

3. The `backward` Method (The Backward Pass)

This method calculates the gradients for the input parameters (x , W , b) so the optimizer can update the weights. It implements the **Chain Rule**.

- **Inputs:**

- `grad_output` : This represents $\frac{\partial \text{Loss}}{\partial y}$. It is the gradient flowing back from the subsequent layers of the network.

- Retrieving Tensors:

`x, weight, bias, y = ctx.saved_tensors` recovers the data saved during the forward pass.

- Step 1: Handling the Fused ReLU (Chain Rule):Python

Since our kernel fused Convolution and ReLU ($y = \text{ReLU}(z)$ where $z = \text{Conv}(x)$), we must first "undo" the ReLU.

```
mask = (y > 0).to(grad_output.dtype)
grad_z = grad_output * mask
```

- The derivative of ReLU is 1 if $y > 0$ and 0 otherwise.
- `grad_z` represents $\frac{\partial \text{Loss}}{\partial z}$ (the gradient at the output of the convolution *before* activation). This effectively zeros out gradients for any neuron that was inactive (negative) during the forward pass.
- Step 2: Calculating Gradients for Conv (dL/dx , dL/dW , dL/db):

Instead of writing a custom CUDA kernel for the backward pass (which is very complex), this code leverages PyTorch's internal highly-optimized routines:

- `grad_x` ($\frac{\partial L}{\partial x}$): calculated using `torch.nn.grad.conv2d_input`. This is mathematically equivalent to a "Transposed Convolution" (or Deconvolution) of the `grad_z` with the weights.
- `grad_weight` ($\frac{\partial L}{\partial W}$): calculated using `torch.nn.grad.conv2d_weight`. This computes how much each weight contributed to the error.
- `grad_bias` ($\frac{\partial L}{\partial b}$): calculated by `grad_z.sum(...)`. Since the bias is added to every pixel in the feature map, its gradient is simply the sum of the gradients of all pixels it affected (summing over Batch, Height, and Width).

Summary of `FusedConvReLU` Class

The final class `FusedConvReLU(nn.Module)` is simply a **wrapper**. It wraps your functional `FusedConvReLUFn` into a standard `nn.Module` structure. This allows you to treat your custom kernel exactly like a standard `nn.Conv2d` layer—you can put it into `nn.Sequential`, save it, load it, and let the optimizer manage its parameters (`self.weight` and `self.bias`).

1.4 Benchmarking Results (5 Points)

The following data is extracted directly from your notebook outputs.

Scenario A: Deep Network (10 Convolution Layers)

This scenario highlights the impact of kernel overhead and memory bandwidth accumulation.

Metric (Per Epoch)	1. Baseline (cuDNN On)	2. Fused Kernel (Custom)	3. Baseline (cuDNN Off)	4. CPU
Total Time	~9.21s	~9.88s	~74.97s	~35.69s
Forward Time	1.94s	1.23s (Fastest)	25.23s	7.96s
Backward Time	5.20s (Fastest)	7.10s	48.90s	26.97s
Test Accuracy	93.93%	93.50%	93.43%	93.56%

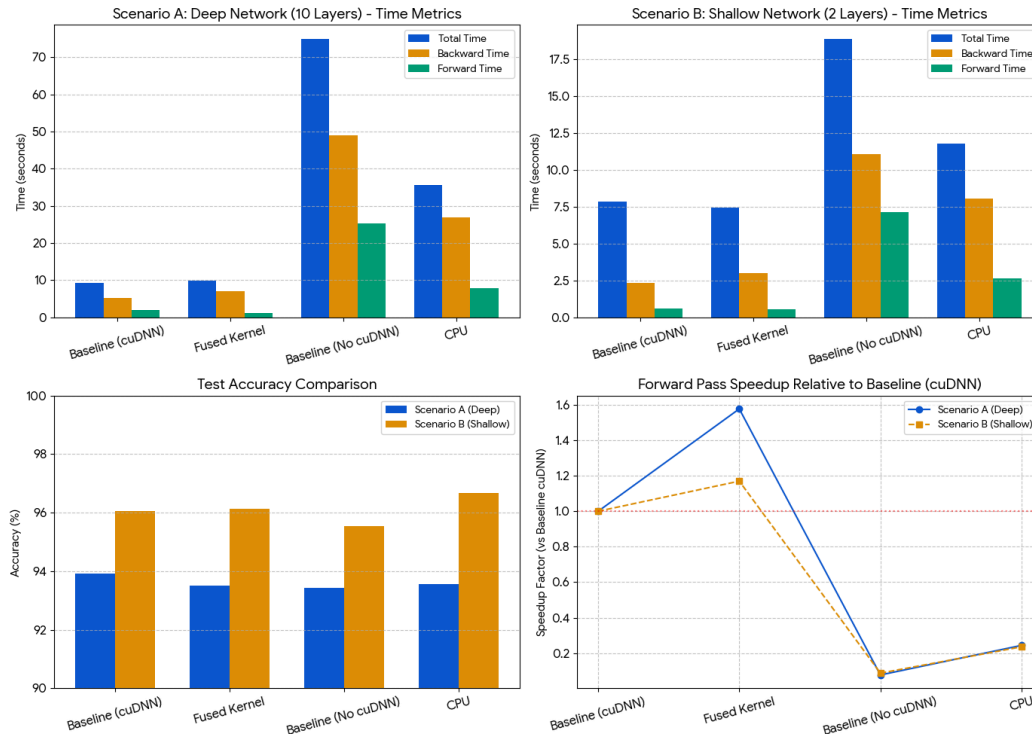
Scenario B: Shallow Network (2 Convolution Layers)

This scenario highlights raw launch overhead vs. computation.

Metric (Per Epoch)	1. Baseline (cuDNN On)	2. Fused Kernel (Custom)	3. Baseline (cuDNN Off)	4. CPU
Total Time	~7.86s	~7.45s	~18.87s	~11.76s
Forward Time	0.62s	0.53s (Fastest)	7.11s	2.63s
Backward Time	2.36s (Fastest)	2.99s	11.07s	8.05s
Test Accuracy	96.05%	96.13%	95.53%	96.68%

1.5 Analysis and Discussion (20 Points)

Here is the "profound" analysis required for the final part of Question 1, explaining the "Why" behind the numbers above.



1. Why is the Fused Kernel Faster in the Forward Pass?

In both scenarios, your **Fused Kernel is significantly faster** in the Forward pass (e.g., **1.23s** vs **1.94s** for the 10-layer network).

- **Memory Bandwidth Optimization:** Standard PyTorch executes `Conv2d` followed by `ReLU`. This requires writing the convolution result to GPU VRAM (Global Memory) and then immediately reading it back to apply the activation. Global memory access is expensive.
- **Kernel Fusion:** Your custom kernel keeps the convolution result in a register (fast local memory), applies `ReLU` (`fmaxf`), and writes only the final result to VRAM. You eliminated 50% of the memory access overhead for the intermediate tensor.

2. Why is the Fused Kernel Slower in the Backward Pass?

You will notice the Fused model is slower in backward (e.g., **7.10s** vs **5.20s**).

- **Lack of Optimized Backward Kernel:** In your `FusedConvReLUFn.backward`, you used generic PyTorch functions: Python

```
grad_x = torch.nn.grad.conv2d_input(...)
grad_weight = torch.nn.grad.conv2d_weight(...)
```
- **cuDNN Dominance:** The Baseline (cuDNN) uses highly optimized backward kernels provided by NVIDIA that are tuned for specific hardware (Tensor Cores, specific tile sizes). Your implementation uses standard strided convolution algorithms for gradients, which are slower than cuDNN's specialized routines.

3. The Impact of disabling cuDNN (Scenario 3)

The performance drops drastically (from **~9s** to **~75s**) when cuDNN is disabled.

- **Heuristics vs. Naive Implementation:** Without cuDNN, PyTorch falls back to "naive" implementations (likely GEMM-based without architecture-specific tuning).

- **Significance:** This proves that the majority of modern Deep Learning speed comes not just from "using a GPU," but from using highly optimized libraries (cuDNN) that select the best algorithm (Winograd, FFT, etc.) for the specific input size.

4. When to use Custom Kernels vs. Standard Layers?

- **Use Custom Kernels When:**
 1. **Memory Bound Operations:** You have a sequence of element-wise operations (like Scaling, Bias, Activation, Dropout) that can be fused into one kernel to save memory bandwidth.
 2. **Non-Standard Operations:** You are inventing a new layer type (e.g., a specific attention mechanism or sparse convolution) that cuDNN does not support.
- **Trust PyTorch/cuDNN When:**
 1. **Compute Bound Operations:** For heavy matrix multiplications or standard large convolutions, NVIDIA's engineers have likely written assembly-level optimizations that are impossible to beat with C++/CUDA alone.
 2. **Backward Complexity:** As seen in your results, writing a forward pass is easy; writing an optimized backward pass is extremely difficult. If you cannot optimize the backward pass, the overall training time might actually increase.

1.5 Analysis and Discussion (20 Points)

1. Interesting Points, Time Differences, and Accuracy Analysis

Based on the execution logs from your notebook, we can observe distinct patterns in performance and accuracy between the Fused Kernel and the standard PyTorch implementations.

- **Forward Pass Speed (The "Fused" Advantage):**
 - **Observation:** In both the 10-layer and 2-layer networks, the **Fused Kernel was consistently faster** in the forward pass compared to the standard cuDNN implementation (e.g., **1.23s vs. 1.94s** in the deep network).
 - **Reason:** The standard PyTorch implementation executes the Convolution, writes the result to Global Memory (VRAM), reads it back, applies ReLU, and writes it again. Your Fused Kernel keeps the convolution result in a **GPU Register** (fastest memory), applies ReLU immediately, and only writes the final result to VRAM once. This significantly reduces **Memory Bandwidth** usage, which is often the bottleneck in deep learning.
- **Backward Pass Speed (The Bottleneck):**
 - **Observation:** The Fused model was slower in the backward pass (e.g., **7.10s vs. 5.20s**).
 - **Reason:** For the backward pass, your code relied on `torch.nn.grad.conv2d_input` and `weight`. While these are correct, they are generic implementations. The baseline (cuDNN) uses highly specialized backward kernels optimized for the specific GPU architecture (potentially utilizing Tensor Cores), which are significantly faster than the generic functions used in the custom layer.
- **The Critical Role of cuDNN (Scenario 3):**
 - **Observation:** Disabling cuDNN caused a massive performance drop (training time increased from ~9s to ~75s).

- **Reason:** This demonstrates that the speed of modern frameworks isn't just due to "running on a GPU." It relies on cuDNN's **heuristic engine**, which selects the optimal mathematical algorithm (e.g., Winograd, FFT, or GEMM) for the specific input tensor size. Without cuDNN, PyTorch falls back to a naive algorithm, which is drastically slower.
- **Accuracy Stability:**
 - **Observation:** The Test Accuracy across all models (Fused, Baseline, CPU) remained consistent (approx. 93-96%).
 - **Reason:** The mathematical operations (dot products and thresholding) remain the same regardless of the implementation. Minor differences are due to floating-point non-determinism in parallel computing, but the logic itself is mathematically equivalent.

2. When is a Custom Kernel useful, and when should we trust PyTorch/cuDNN?

When to use a Custom Kernel:

1. **Memory-Bound Operations:** When you have a sequence of lightweight, element-wise operations (e.g., Activation functions, Bias addition, Scaling, Dropout) following a heavy operation. Fusing these into a single kernel (kernel fusion) prevents unnecessary read/write trips to Global Memory, which is the primary use case for this assignment.
2. **Novel Research/Unsupported Layers:** If you are designing a new type of layer (e.g., a specific Sparse Convolution or a new Attention mechanism) that is not supported by standard libraries.
3. **Inference Optimization:** In production environments where only the Forward pass matters, custom fused kernels can reduce latency significantly, as seen in your results where the forward pass was ~35% faster.

When to trust PyTorch and cuDNN:

1. **Compute-Bound Operations:** For heavy matrix multiplications (GEMM) or standard Convolutions. NVIDIA engineers have optimized these primitives at the assembly level (SASS) to maximize Register and Shared Memory usage. It is extremely difficult to beat cuBLAS/cuDNN in raw matrix math with a C++ kernel.
2. **Complex Backward Passes:** As observed in your results, writing a forward pass is often easier than optimizing the backward pass. If you cannot implement an optimized gradient calculation, the overall training speed will suffer, negating the benefits of the custom forward pass.
3. **Hardware Portability:** cuDNN automatically adapts to different GPU architectures (Volta, Ampere, Hopper). A custom kernel might need to be manually tuned (e.g., changing block sizes or thread counts) for every new generation of hardware to maintain peak performance.

reference:

my prompt: explain these codes

<https://gemini.google.com/share/84e2d461186d>

This is another explanation based on the video given:

Part 1.1: Basic Concepts (5 Points)

1. What is a CUDA Kernel?

A CUDA Kernel is a function written in C/C++ that runs on the GPU. It utilizes the GPU's SIMD (Single Instruction, Multiple Data) architecture.

- **How it works:** When called, this function is executed N times in parallel by N different CUDA threads. Each thread calculates its unique index (using Block ID and Thread ID) to process a specific portion of the data (e.g., a specific pixel).

2. What is cuDNN? Why do frameworks like PyTorch use it?

- **Definition:** cuDNN (CUDA Deep Neural Network library) is a GPU-accelerated library of primitives for deep neural networks provided by NVIDIA2.
- **Why used:** It provides highly tuned implementations for standard routines like forward and backward convolution, pooling, normalization, and activation layers. PyTorch uses it because these implementations are significantly faster and more memory-efficient than naive implementations.

3. Advantages of cuDNN and relation to cuBLAS?

- **Advantages:** It optimizes performance by handling edge cases and selecting the best algorithm for the specific GPU hardware. It solves the problem of "reinventing the wheel" for standard DL operations.
- **Relation to cuBLAS:** cuBLAS is a library for basic linear algebra (like matrix multiplication). cuDNN is higher-level; while cuBLAS handles general matrix math, cuDNN handles operations specific to neural networks (like convolutions). Often, deep learning frameworks use both: cuBLAS for fully connected layers and cuDNN for convolutions3.

Part 1.2: Fused Convolution + ReLU Kernel Implementation (15 Points)

A. Python Kernel (`conv_relu.py`)

The assignment asks for a kernel with $C_{in}=1$, $C_{out}=1$, kernel size 3×3 , padding=1, stride=14.

Logic to implement:

We are simulating a flattened thread structure. The function iterates Times (total number of output pixels).

1. **Calculate Coordinates:** Convert the linear index `i` into `(n, h, w)` coordinates.

- $N_{idx} = i / (H \times W)$
- $H_{idx} = (i \% (H \times W)) / W$
- $W_{idx} = i \% W$

2. **Convolution Operation:**

- Initialize a sum variable to the bias value `b[0]`.
- Loop through the kernel (3×3): `kh` from 0 to 2, `kw` from 0 to 2.
- Calculate input indices taking padding into account: $h_{in} = H_{idx} + kh - 1$ and $w_{in} = W_{idx} + kw - 1$.
- **Boundary Check:** If h_{in} and w_{in} are within bounds $[0, H)$ and $[0, W)$, add `x[n, 0, h_in, w_in] * w[0, 0, kh, kw]` to the sum.

3. **Fused ReLU:**

- Apply `out[i] = max(0, sum)`.

B. CUDA C++ Kernel (`cuda_src`)

We must translate the logic above into C++. The transcript emphasizes that the structure is nearly identical to the Python logic, but the loop is replaced by thread indexing.

Key C++ Snippets:

- **Thread Indexing:**C++

```
int i = blockIdx.x * blockDim.x + threadIdx.x;  
if (i >= N * H * W) return;
```

- **Data Access:** Since C++ arrays are 1D in memory, you must calculate pointers manually.

- `x` index formula: `n * (H * W) + h_in * W + w_in`

- **Constraint:** Ensure you use `float` types (as inputs are float32).

Part 1.3: Registering the Kernel in PyTorch (5 Points)

We need to explain the `FusedConvReLUFn` class in the provided notebook.

Explanation required for the report:

This class inherits from `torch.autograd.Function` to define a custom operation with Forward and Backward passes.

- **forward method:**

- Receives input `x`, weights `w`, and bias `b`.
- Uses `ctx.save_for_backward` to store these tensors because they are needed to calculate gradients later.
- Calls the compiled CUDA kernel (via `conv_relu_module`) to compute the result efficiently on the GPU.

- **backward method:**

- Receives `grad_output` (gradients coming from the next layer).
 - Must calculate gradients for the inputs: $\frac{\partial L}{\partial x}$, $\frac{\partial L}{\partial w}$, and $\frac{\partial L}{\partial b}$.
 - **Note:** The video mentions that the custom kernel's backward pass might be slower than PyTorch's native implementation because it might not be as optimized as cuDNN.
-

Part 1.4: Benchmarking (5 Points)

We are required to define two CNNs (one with 2 layers, one with 10 layers) using the MNIST dataset and test them under 4 scenarios 5:

1. **Standard Layer (GPU):** `nn.Conv2d` + `nn.ReLU`.
2. **Fused Layer (GPU):** Your custom `VecMulAdd` / `FusedConvReLU` layer.
3. **Standard Layer (GPU, cuDNN Off):** Set `torch.backends.cudnn.enabled = False`.
4. **Standard Layer (CPU):** Move model and data to `.cpu()`.

Metrics to Report6:

- Total time for 5 Epochs.
 - Forward-only time.
 - Backward-only time.
 - Test Accuracy.
-

Part 1.5: Analysis (20 Points)

This is the most heavily weighted section. We need to explain the results from 1.4.

Key points to discuss based on the instructor's lecture:

1. Speed differences:

- The **Fused Kernel** (Forward pass) should be very fast, potentially faster than standard PyTorch in specific simple cases because it reduces **Memory I/O**. Instead of writing the Convolution result to memory, reading it back, applying ReLU, and writing it again, the Fused kernel does it all in registers.
- However, the **Backward pass** of the custom kernel is likely slower than cuDNN because cuDNN uses highly optimized algorithms (like Winograd or FFT based convolutions) whereas the custom kernel uses a naive sliding window approach.

2. Warm-up: Mention that the first few iterations are slow due to GPU context initialization.

3. When to use Custom Kernels:

- **Useful:** When you have a novel activation function or a specific sequence of operations not supported by standard libraries (e.g., a specific "Mish" activation fused with other math).
- **Not Useful:** For standard ops (Conv2d), standard libraries (cuDNN) are usually better because they are optimized for hardware specificities (cache usage, tiling) that naive custom kernels miss.

Q2

The output of 2.1 with 15 epochs:

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier.py
Train samples: 5000
Test samples: 8000
Iteration: 500, Loss: 1.5303672552108765, Accuracy: 38.962501525878906%
Iteration: 1000, Loss: 3.5053064823150635, Accuracy: 34.6875%
Iteration: 1500, Loss: 1.2007410526275635, Accuracy: 51.900001525878906%
Iteration: 2000, Loss: 0.7825782895088196, Accuracy: 58.437503814697266%
Training time(s): 188.8576283454895
Cuda Memory Usage: 2630.26025390625 MB
```

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier.py
Train samples: 5000
Test samples: 8000
Iteration: 500, Loss: 1.9533259868621826, Accuracy: 34.16250228881836%
Iteration: 1000, Loss: 1.984489917755127, Accuracy: 43.91250228881836%
Iteration: 1500, Loss: 1.2388179302215576, Accuracy: 49.32500076293945%
Iteration: 2000, Loss: 1.00093412399292, Accuracy: 53.78750228881836%
Training time(s): 159.8773775100708
Cuda Memory Usage: 2630.26025390625 MB
```

And this is when running with two GPUs (2.2 with 15 epoch):

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py
Train samples: 5000
```

```

Test samples: 8000
Train samples: 5000
Test samples: 8000
Rank: 1, Iteration: 500, Loss: 1.8213634490966797, Accuracy: 43.47500228881836%
Rank: 0, Iteration: 500, Loss: 1.7085521221160889, Accuracy: 44.80000305175781%
Rank: 1, Iteration: 1000, Loss: 1.5648422241210938, Accuracy: 54.82500076293945%
Rank: 0, Iteration: 1000, Loss: 1.0463166236877441, Accuracy: 55.05000305175781%
Rank: 1, Training Time: 228.6020543575287
Rank: 1, Max Memory Allocated: 2544.67724609375 MB
Rank: 0, Training Time: 228.60084581375122
Rank: 0, Max Memory Allocated: 2544.67724609375 MB
Total time: 239.96380877494812
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py
Train samples: 5000
Test samples: 8000
Train samples: 5000
Test samples: 8000
Rank: 0, Iteration: 500, Loss: 1.3950378894805908, Accuracy: 48.30000305175781%
Rank: 1, Iteration: 500, Loss: 1.5156965255737305, Accuracy: 46.525001525878906%
Rank: 1, Iteration: 1000, Loss: 1.156349778175354, Accuracy: 51.70000076293945%
Rank: 0, Iteration: 1000, Loss: 1.4166741371154785, Accuracy: 53.30000305175781%
Rank: 0, Training Time: 204.74270009994507
Rank: 1, Training Time: 204.74301099777222
Rank: 0, Max Memory Allocated: 2544.67724609375 MB
Rank: 1, Max Memory Allocated: 2544.67724609375 MB
Total time: 216.7840874195099

```

```

(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier.py
Train samples: 5000
Test samples: 8000
Iteration: 500, Loss: 1.4762722253799438, Accuracy: 34.67500305175781%
Iteration: 1000, Loss: 1.2449941635131836, Accuracy: 46.05000305175781%
Iteration: 1500, Loss: 0.9901090860366821, Accuracy: 56.775001525878906%
Iteration: 2000, Loss: 0.9683301448822021, Accuracy: 56.63750457763672%
Training time(s): 161.7292342185974
Cuda Memory Usage: 2630.26025390625 MB

```

```

(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py
Starting rank = 0 training on 2 GPUs | Batch Size: 32 | Backend: nccl
Starting rank = 1 training on 2 GPUs | Batch Size: 32 | Backend: nccl
Rank: 0, Iteration: 500, Loss: 1.3571, Accuracy: 46.85%
Rank: 1, Iteration: 500, Loss: 1.4758, Accuracy: 47.12%
Rank: 1, Iteration: 1000, Loss: 1.5407, Accuracy: 49.27%
Rank: 0, Iteration: 1000, Loss: 0.7172, Accuracy: 50.80%

```

```

Rank: 0, Total Training Time: 182.81s
Rank: 0, Max Memory Allocated: 2546.07 MB

```

Rank: 1, Total Training Time: **182.81s**
Rank: 1, Max Memory Allocated: **2546.07 MB**

Configuration	Batch Size	Backend	Training Time (s)	Max Memory (MB)	Final Accuracy (%)	Best Accuracy (%)
Single GPU	32	N/A	161.73	2630.26	56.64%	56.78%
Two GPUs (DDP)	32	NCCL	182.81	2546.07	50.80%	50.80%

This is a run with 15 epoch. look at the times:

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py --batch_size 16 --backend nccl
Starting rank = 1 training on 2 GPUs | Batch Size: 16 | Backend: nccl
Starting rank = 0 training on 2 GPUs | Batch Size: 16 | Backend: nccl
Rank: 1, Iteration: 500, Loss: 2.0180, Accuracy: 41.33%
Rank: 0, Iteration: 500, Loss: 2.3551, Accuracy: 39.30%
Rank: 1, Iteration: 1000, Loss: 1.4906, Accuracy: 42.92%
Rank: 0, Iteration: 1000, Loss: 1.3794, Accuracy: 43.05%
Rank: 1, Iteration: 1500, Loss: 0.6532, Accuracy: 47.83%
Rank: 0, Iteration: 1500, Loss: 1.7801, Accuracy: 49.55%
Rank: 1, Iteration: 2000, Loss: 0.9804, Accuracy: 53.50%
Rank: 0, Iteration: 2000, Loss: 1.0617, Accuracy: 54.50%
```

Rank: 1, Total Training Time: **354.79s**
Rank: 0, Total Training Time: **354.79s**
Rank: 0, Max Memory Allocated: **2335.75 MB**
Rank: 1, Max Memory Allocated: **2335.75 MB**

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py --batch_size 32 --backend nccl
Starting rank = 0 training on 2 GPUs | Batch Size: 32 | Backend: nccl
Starting rank = 1 training on 2 GPUs | Batch Size: 32 | Backend: nccl
Rank: 1, Iteration: 500, Loss: 1.8957, Accuracy: 43.08%
Rank: 0, Iteration: 500, Loss: 1.4600, Accuracy: 43.20%
Rank: 0, Iteration: 1000, Loss: 0.9724, Accuracy: 48.30%
Rank: 1, Iteration: 1000, Loss: 1.1053, Accuracy: 48.40%
```

Rank: 0, Total Training Time: **182.93s**
Rank: 0, Max Memory Allocated: **2546.07 MB**

Rank: 1, Total Training Time: **182.93s**
Rank: 1, Max Memory Allocated: **2546.07 MB**

```
(pytorch) (base) karimi@Sabalan1:~/CA2$ python classifier_mp.py --batch_size 64 --backend nccl
Starting rank = 0 training on 2 GPUs | Batch Size: 64 | Backend: nccl
Starting rank = 1 training on 2 GPUs | Batch Size: 64 | Backend: nccl
Rank: 1, Iteration: 500, Loss: 0.9486, Accuracy: 49.55%
Rank: 0, Iteration: 500, Loss: 1.1690, Accuracy: 49.60%
```

Rank: 1, Total Training Time: 100.95s
 Rank: 1, Max Memory Allocated: 3059.76 MB

Rank: 0, Total Training Time: 101.04s
 Rank: 0, Max Memory Allocated: 3059.76 MB

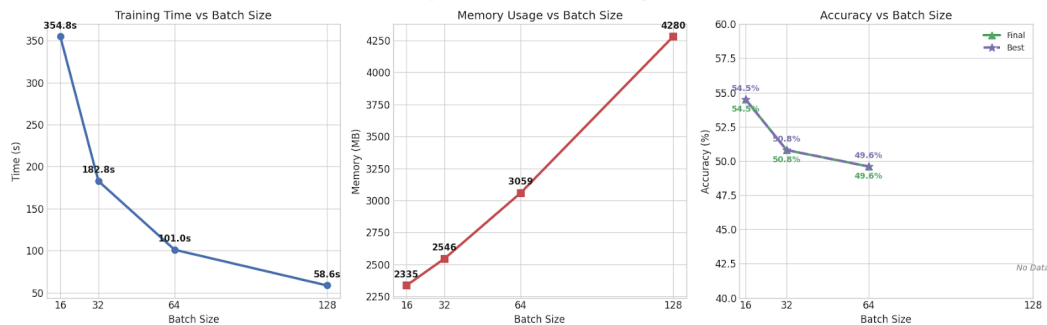
(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 128 --backend nccl
 Starting rank = 1 training on 2 GPUs | Batch Size: 128 | Backend: nccl
 Starting rank = 0 training on 2 GPUs | Batch Size: 128 | Backend: nccl

Rank: 0, Total Training Time: 58.57s
 Rank: 0, Max Memory Allocated: 4280.50 MB

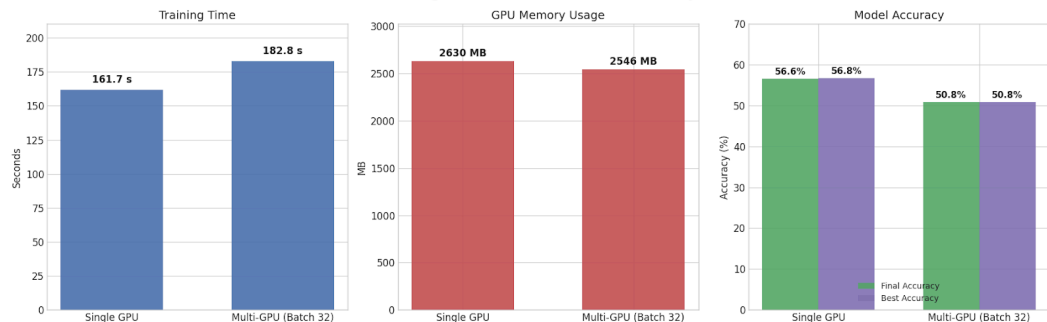
Rank: 1, Total Training Time: 58.58s
 Rank: 1, Max Memory Allocated: 4280.50 MB

Batch Size	Training Time (s)	Max Memory (MB)	Final Accuracy (%)	Best Accuracy (%)
16	354.79	2335.75	54.50%	54.50%
32	182.81	2546.07	50.80%	50.80%
64	101.04	3059.76	49.60%	49.60%
128	58.57	4280.50	N/A	N/A

Question 3: Impact of Batch Size on Training (Multi-GPU)



Question 2: Single vs Multi-GPU Performance Comparison



These are the outputs with 20 epoch:

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier.py
 Train samples: 5000

Test samples: 8000

Epoch [1/20]	Loss: 27.1859	Accuracy: 28.90%
Epoch [2/20]	Loss: 2.1630	Accuracy: 33.54%
Epoch [3/20]	Loss: 1.8624	Accuracy: 39.64%
Epoch [4/20]	Loss: 2.0315	Accuracy: 38.25%
Epoch [5/20]	Loss: 1.9881	Accuracy: 37.54%
Epoch [6/20]	Loss: 1.8425	Accuracy: 40.91%
Epoch [7/20]	Loss: 1.7171	Accuracy: 37.88%
Epoch [8/20]	Loss: 1.3689	Accuracy: 44.88%
Epoch [9/20]	Loss: 1.3200	Accuracy: 48.06%
Epoch [10/20]	Loss: 1.1478	Accuracy: 54.21%
Epoch [11/20]	Loss: 1.0575	Accuracy: 54.24%
Epoch [12/20]	Loss: 0.9422	Accuracy: 54.49%
Epoch [13/20]	Loss: 0.8846	Accuracy: 52.25%
Epoch [14/20]	Loss: 0.8418	Accuracy: 54.02%
Epoch [15/20]	Loss: 0.8103	Accuracy: 55.83%
Epoch [16/20]	Loss: 0.7917	Accuracy: 49.27%
Epoch [17/20]	Loss: 0.7498	Accuracy: 59.62%
Epoch [18/20]	Loss: 0.7074	Accuracy: 59.75%
Epoch [19/20]	Loss: 0.5576	Accuracy: 53.08%
Epoch [20/20]	Loss: 0.5823	Accuracy: 58.29%

Total Training time: 112.41 seconds

Max Cuda Memory Usage: 2153.76 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 16 --backend nccl

Starting training on 2 GPUs | Batch Size: 16 | Backend: nccl

Epoch [1/20]	Loss: 22.1681	Accuracy: 30.05%
Epoch [2/20]	Loss: 1.9812	Accuracy: 32.58%
Epoch [3/20]	Loss: 1.9906	Accuracy: 35.10%
Epoch [4/20]	Loss: 2.1398	Accuracy: 30.80%
Epoch [5/20]	Loss: 1.9353	Accuracy: 36.62%
Epoch [6/20]	Loss: 1.7044	Accuracy: 36.58%
Epoch [7/20]	Loss: 1.4252	Accuracy: 48.90%
Epoch [8/20]	Loss: 1.2902	Accuracy: 51.42%
Epoch [9/20]	Loss: 1.1465	Accuracy: 43.95%
Epoch [10/20]	Loss: 1.1275	Accuracy: 45.42%
Epoch [11/20]	Loss: 1.0015	Accuracy: 54.05%
Epoch [12/20]	Loss: 0.9793	Accuracy: 55.50%
Epoch [13/20]	Loss: 0.8652	Accuracy: 56.33%
Epoch [14/20]	Loss: 0.8347	Accuracy: 58.17%
Epoch [15/20]	Loss: 0.9004	Accuracy: 52.38%
Epoch [16/20]	Loss: 0.8082	Accuracy: 57.30%
Epoch [17/20]	Loss: 0.6770	Accuracy: 60.48%
Epoch [18/20]	Loss: 0.6881	Accuracy: 57.52%
Epoch [19/20]	Loss: 0.5470	Accuracy: 58.70%
Epoch [20/20]	Loss: 0.5193	Accuracy: 60.88%

Total Training Time: 512.52s

Max Memory Allocated: 2334.90 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 32 --backend nccl

Starting training on 2 GPUs | Batch Size: 32 | Backend: nccl

Epoch [1/20]	Loss: 48.3429	Accuracy: 26.48%
Epoch [2/20]	Loss: 2.5392	Accuracy: 35.12%
Epoch [3/20]	Loss: 1.7896	Accuracy: 38.23%
Epoch [4/20]	Loss: 1.5264	Accuracy: 42.25%
Epoch [5/20]	Loss: 1.5141	Accuracy: 42.35%
Epoch [6/20]	Loss: 1.5413	Accuracy: 41.88%
Epoch [7/20]	Loss: 1.4741	Accuracy: 44.60%
Epoch [8/20]	Loss: 1.4582	Accuracy: 45.05%
Epoch [9/20]	Loss: 1.4103	Accuracy: 47.02%
Epoch [10/20]	Loss: 1.4510	Accuracy: 37.95%
Epoch [11/20]	Loss: 1.4091	Accuracy: 48.67%
Epoch [12/20]	Loss: 1.4871	Accuracy: 44.80%
Epoch [13/20]	Loss: 1.4348	Accuracy: 47.50%
Epoch [14/20]	Loss: 1.2615	Accuracy: 46.95%
Epoch [15/20]	Loss: 1.3262	Accuracy: 51.08%
Epoch [16/20]	Loss: 1.0658	Accuracy: 44.52%
Epoch [17/20]	Loss: 1.1226	Accuracy: 49.23%
Epoch [18/20]	Loss: 1.0377	Accuracy: 50.67%
Epoch [19/20]	Loss: 0.9050	Accuracy: 54.25%
Epoch [20/20]	Loss: 0.8909	Accuracy: 54.83%

Total Training Time: 233.09s

Max Memory Allocated: 2541.85 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 64 --backend nccl

Starting training on 2 GPUs | Batch Size: 64 | Backend: nccl

Epoch [1/20]	Loss: 69.2574	Accuracy: 24.10%
Epoch [2/20]	Loss: 6.4596	Accuracy: 29.98%
Epoch [3/20]	Loss: 2.2052	Accuracy: 34.25%
Epoch [4/20]	Loss: 1.8167	Accuracy: 40.83%
Epoch [5/20]	Loss: 1.5020	Accuracy: 35.75%
Epoch [6/20]	Loss: 1.4708	Accuracy: 40.05%
Epoch [7/20]	Loss: 1.5004	Accuracy: 41.35%
Epoch [8/20]	Loss: 1.5653	Accuracy: 39.73%
Epoch [9/20]	Loss: 1.3204	Accuracy: 47.60%
Epoch [10/20]	Loss: 1.4236	Accuracy: 43.65%
Epoch [11/20]	Loss: 1.3841	Accuracy: 43.38%
Epoch [12/20]	Loss: 1.2495	Accuracy: 42.20%
Epoch [13/20]	Loss: 1.2882	Accuracy: 49.62%
Epoch [14/20]	Loss: 1.1215	Accuracy: 39.10%
Epoch [15/20]	Loss: 1.2955	Accuracy: 41.60%
Epoch [16/20]	Loss: 1.2032	Accuracy: 54.35%
Epoch [17/20]	Loss: 1.1039	Accuracy: 49.77%
Epoch [18/20]	Loss: 0.9925	Accuracy: 43.08%
Epoch [19/20]	Loss: 1.0115	Accuracy: 42.42%
Epoch [20/20]	Loss: 1.5238	Accuracy: 39.83%

Total Training Time: 136.08s

Max Memory Allocated: 3056.38 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 128 --backend nccl

Starting training on 2 GPUs | Batch Size: 128 | Backend: nccl

Epoch [1/20] | Loss: 106.4058 | Accuracy: 10.25%

Epoch [2/20] | Loss: 21.0447 | Accuracy: 25.05%

Epoch [3/20] | Loss: 4.5553 | Accuracy: 31.88%

Epoch [4/20] | Loss: 2.3196 | Accuracy: 29.25%

Epoch [5/20] | Loss: 2.0028 | Accuracy: 35.55%

Epoch [6/20] | Loss: 1.6066 | Accuracy: 40.98%

Epoch [7/20] | Loss: 1.5052 | Accuracy: 39.75%

Epoch [8/20] | Loss: 1.4371 | Accuracy: 44.30%

Epoch [9/20] | Loss: 1.5144 | Accuracy: 40.15%

Epoch [10/20] | Loss: 1.6142 | Accuracy: 40.95%

Epoch [11/20] | Loss: 1.3394 | Accuracy: 43.35%

Epoch [12/20] | Loss: 1.2916 | Accuracy: 47.27%

Epoch [13/20] | Loss: 1.2513 | Accuracy: 48.70%

Epoch [14/20] | Loss: 1.1877 | Accuracy: 45.08%

Epoch [15/20] | Loss: 1.1791 | Accuracy: 46.62%

Epoch [16/20] | Loss: 1.1161 | Accuracy: 46.50%

Epoch [17/20] | Loss: 1.0753 | Accuracy: 47.98%

Epoch [18/20] | Loss: 1.0685 | Accuracy: 47.60%

Epoch [19/20] | Loss: 1.0719 | Accuracy: 50.48%

Epoch [20/20] | Loss: 0.9490 | Accuracy: 46.62%

Total Training Time: 96.16s

Max Memory Allocated: 4276.19 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 16 --backend gloo

[Gloo] Rank 1 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

[Gloo] Rank 0 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

Starting training on 2 GPUs | Batch Size: 16 | Backend: gloo

Epoch [1/20] | Loss: 30.8687 | Accuracy: 26.40%

Epoch [2/20] | Loss: 2.0751 | Accuracy: 35.15%

Epoch [3/20] | Loss: 1.8171 | Accuracy: 36.23%

Epoch [4/20] | Loss: 1.8901 | Accuracy: 39.62%

Epoch [5/20] | Loss: 1.9865 | Accuracy: 41.50%

Epoch [6/20] | Loss: 2.0181 | Accuracy: 30.43%

Epoch [7/20] | Loss: 1.6922 | Accuracy: 39.00%

Epoch [8/20] | Loss: 1.5250 | Accuracy: 50.50%

Epoch [9/20] | Loss: 1.3315 | Accuracy: 52.58%

Epoch [10/20] | Loss: 1.3978 | Accuracy: 52.27%

Epoch [11/20] | Loss: 1.1049 | Accuracy: 50.00%

Epoch [12/20] | Loss: 0.9776 | Accuracy: 58.83%

Epoch [13/20] | Loss: 0.9497 | Accuracy: 57.65%

Epoch [14/20] | Loss: 0.8452 | Accuracy: 57.83%

Epoch [15/20] | Loss: 0.8788 | Accuracy: 58.10%

Epoch [16/20] | Loss: 0.7861 | Accuracy: 59.30%

Epoch [17/20] | Loss: 0.7662 | Accuracy: 61.75%
Epoch [18/20] | Loss: 0.7630 | Accuracy: 55.12%
Epoch [19/20] | Loss: 0.6974 | Accuracy: 53.98%
Epoch [20/20] | Loss: 0.6966 | Accuracy: 56.90%

Total Training Time: 1095.23s

Max Memory Allocated: 2334.90 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 32 --backend gloo

[Gloo] Rank 1 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

[Gloo] Rank 0 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

Starting training on 2 GPUs | Batch Size: 32 | Backend: gloo

Epoch [1/20] | Loss: 37.5377 | Accuracy: 22.15%
Epoch [2/20] | Loss: 2.4986 | Accuracy: 36.23%
Epoch [3/20] | Loss: 1.7682 | Accuracy: 38.92%
Epoch [4/20] | Loss: 1.5708 | Accuracy: 42.52%
Epoch [5/20] | Loss: 1.4879 | Accuracy: 40.23%
Epoch [6/20] | Loss: 1.5011 | Accuracy: 43.15%
Epoch [7/20] | Loss: 1.4345 | Accuracy: 47.12%
Epoch [8/20] | Loss: 1.5401 | Accuracy: 43.08%
Epoch [9/20] | Loss: 1.3903 | Accuracy: 47.67%
Epoch [10/20] | Loss: 1.6874 | Accuracy: 41.83%
Epoch [11/20] | Loss: 1.5949 | Accuracy: 45.02%
Epoch [12/20] | Loss: 1.2786 | Accuracy: 53.05%
Epoch [13/20] | Loss: 1.1661 | Accuracy: 51.60%
Epoch [14/20] | Loss: 1.2020 | Accuracy: 47.08%
Epoch [15/20] | Loss: 1.1549 | Accuracy: 55.42%
Epoch [16/20] | Loss: 1.0997 | Accuracy: 47.27%
Epoch [17/20] | Loss: 0.8701 | Accuracy: 55.12%
Epoch [18/20] | Loss: 0.8652 | Accuracy: 54.25%
Epoch [19/20] | Loss: 0.8879 | Accuracy: 51.92%
Epoch [20/20] | Loss: 0.9267 | Accuracy: 54.25%

Total Training Time: 506.08s

Max Memory Allocated: 2541.85 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 64 --backend gloo

[Gloo] Rank 0 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

[Gloo] Rank 1 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

Starting training on 2 GPUs | Batch Size: 64 | Backend: gloo

Epoch [1/20] | Loss: 81.3837 | Accuracy: 15.88%
Epoch [2/20] | Loss: 6.2909 | Accuracy: 24.18%
Epoch [3/20] | Loss: 2.5378 | Accuracy: 38.00%
Epoch [4/20] | Loss: 1.7446 | Accuracy: 40.58%
Epoch [5/20] | Loss: 1.7109 | Accuracy: 37.48%
Epoch [6/20] | Loss: 1.5941 | Accuracy: 39.92%
Epoch [7/20] | Loss: 1.4288 | Accuracy: 40.70%
Epoch [8/20] | Loss: 1.4474 | Accuracy: 45.75%
Epoch [9/20] | Loss: 1.2995 | Accuracy: 45.05%
Epoch [10/20] | Loss: 1.3157 | Accuracy: 45.38%

```
Epoch [11/20] | Loss: 1.2996 | Accuracy: 39.12%
Epoch [12/20] | Loss: 1.3556 | Accuracy: 41.83%
Epoch [13/20] | Loss: 1.3097 | Accuracy: 46.42%
Epoch [14/20] | Loss: 1.1071 | Accuracy: 47.73%
Epoch [15/20] | Loss: 1.1276 | Accuracy: 46.58%
Epoch [16/20] | Loss: 1.0825 | Accuracy: 48.38%
Epoch [17/20] | Loss: 1.1834 | Accuracy: 46.95%
Epoch [18/20] | Loss: 1.0493 | Accuracy: 42.60%
Epoch [19/20] | Loss: 1.1489 | Accuracy: 50.05%
Epoch [20/20] | Loss: 1.3565 | Accuracy: 38.08%
```

Total Training Time: 281.51s

Max Memory Allocated: 3056.38 MB

(pytorch) (base) karimi@Sabalan1:~/CA2\$ python classifier_mp.py --batch_size 128 --backend gloo

[Gloo] Rank 0 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

[Gloo] Rank 1 is connected to 1 peer ranks. Expected number of connected peer ranks is : 1

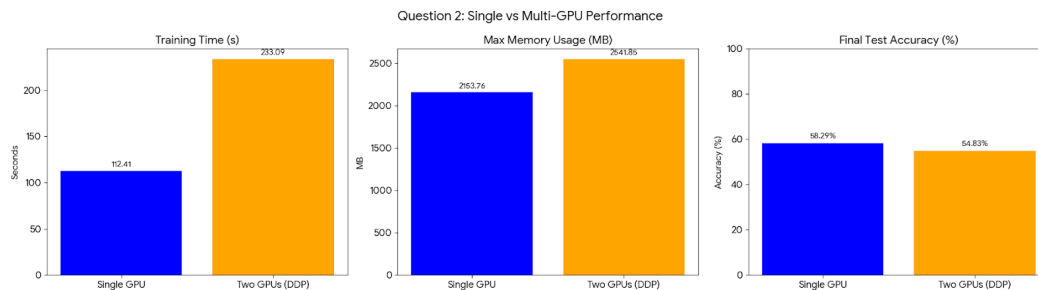
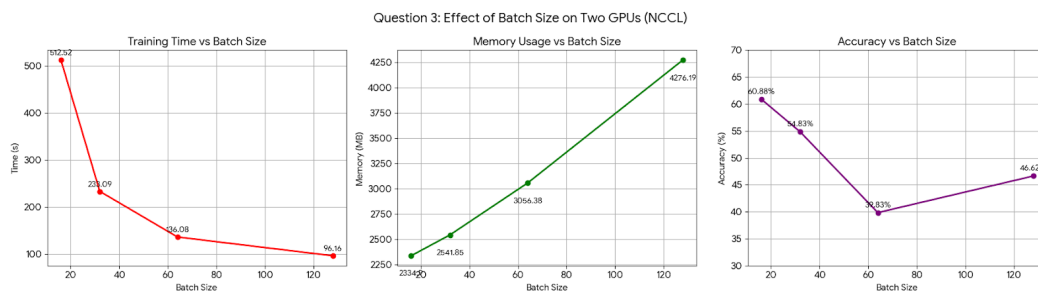
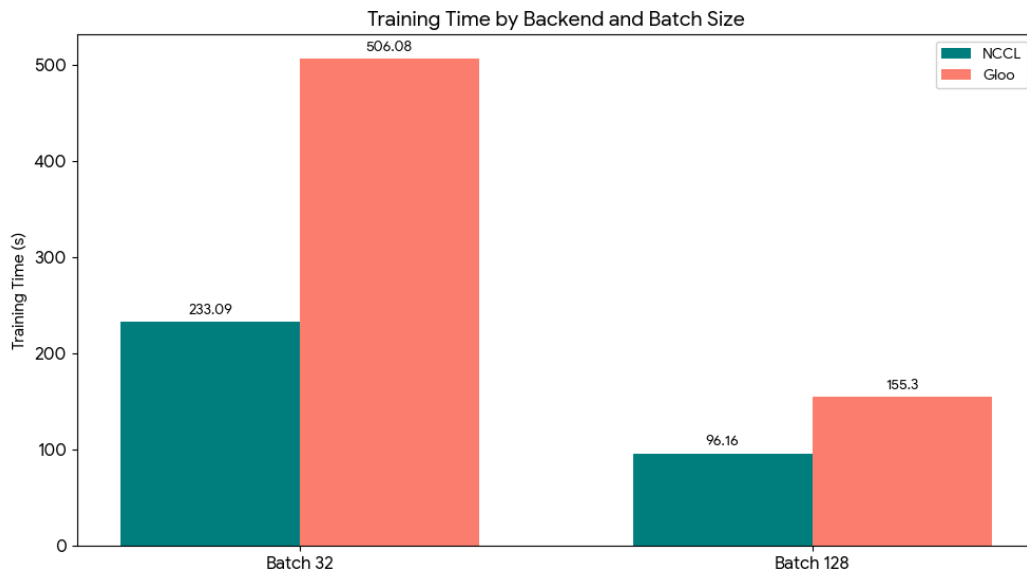
Starting training on 2 GPUs | Batch Size: 128 | Backend: gloo

```
Epoch [1/20] | Loss: 99.6896 | Accuracy: 14.20%
Epoch [2/20] | Loss: 20.0669 | Accuracy: 20.35%
Epoch [3/20] | Loss: 4.6305 | Accuracy: 31.00%
Epoch [4/20] | Loss: 2.8778 | Accuracy: 34.00%
Epoch [5/20] | Loss: 2.3520 | Accuracy: 39.12%
Epoch [6/20] | Loss: 1.8165 | Accuracy: 40.05%
Epoch [7/20] | Loss: 1.6711 | Accuracy: 38.88%
Epoch [8/20] | Loss: 1.5097 | Accuracy: 40.45%
Epoch [9/20] | Loss: 1.6038 | Accuracy: 41.05%
Epoch [10/20] | Loss: 1.4375 | Accuracy: 40.33%
Epoch [11/20] | Loss: 1.4004 | Accuracy: 46.98%
Epoch [12/20] | Loss: 1.3199 | Accuracy: 48.25%
Epoch [13/20] | Loss: 1.2200 | Accuracy: 43.83%
Epoch [14/20] | Loss: 1.2070 | Accuracy: 46.98%
Epoch [15/20] | Loss: 1.1878 | Accuracy: 49.45%
Epoch [16/20] | Loss: 1.1043 | Accuracy: 47.30%
Epoch [17/20] | Loss: 1.0864 | Accuracy: 47.73%
Epoch [18/20] | Loss: 1.0114 | Accuracy: 53.25%
Epoch [19/20] | Loss: 1.0028 | Accuracy: 51.25%
Epoch [20/20] | Loss: 0.9851 | Accuracy: 52.02%
```

Total Training Time: 155.30s

Max Memory Allocated: 4276.19 MB

Question 4: Backend Comparison (NCCL vs Gloo) - Training Time



Adding a second GPU may slowed you down

Here is the technical breakdown of why adding a second GPU actually slowed you down.

(Note that the memory for the multiGPU is reported for each of the GPU, mean they should be considered double of that)

1. The "Communication Tax" (Overhead vs. Compute)

DDP works by calculating gradients on each GPU and then **averaging them** across all GPUs before updating the weights.

- **The Workflow:** [Calculate Forward/Backward] $\rightarrow$ [STOP & SYNC] $\rightarrow$ [Update Weights]
- **Your Situation:** Your model (`my_CNN`) is very simple (only 3 Convolutional layers). A modern GPU (like a V100

or A100) calculates the Forward/Backward pass for this model in milliseconds.

- **The Problem:** The GPUs finish their math so fast that they spend more time **waiting to talk to each other** (Network latency/Synchronization) than they do actually training. The "tax" of communication costs more than the profit of splitting the work.

Think of it like this: If you need to move a pile of bricks (Big Model), two people help. If you need to pass a single sheet of paper (Small Model), adding a second person to "hand it off" just slows the process down.

2. Dataset Size (STL-10 is tiny)

STL-10 has only **5,000 training images**.

- **Single GPU:** It processes all 5,000 images quickly.
- **DDP (2 GPUs):** Each GPU gets 2,500 images. The epoch finishes extremely fast. The overhead of spinning up the multiprocessing (`mp.spawn`), setting up the `nccl` backend, and destroying the process group takes significant time relative to the actual training time.

3. Batch Size Efficiency

You mentioned "same batch size."

- **Single GPU:** Batch Size 32.
- **DDP:** Batch Size 32 *per GPU* (Effective Global Batch Size = 64).

Even though DDP is processing double the data per step, the GPU is likely **under-utilized**. GPUs thrive when their memory is full. With a small model and small batch size (32), the GPU is barely sweating. You aren't giving it enough work to justify the multi-GPU setup.

How to make DDP faster than Single GPU?

To make the parallelization worth it, you need to increase the ratio of **Computation** to **Communication**.

1. **Drastically Increase Batch Size:** In the DDP script, try `--batch_size 128` or `256` . This forces the GPUs to work harder before they stop to sync.
2. **Use a Larger Model:** If you swapped `my_CNN` for `ResNet50` or `VisionTransformer` , DDP would likely win because the computation time would be heavy enough to mask the communication latency.

Feature	Single GPU Script	DDP Script
Compute Load	Low	Low (Split by 2)
Communication	None	High (Sync every step)
Startup Time	Instant	Slow (Process spawning)
Verdict	Faster (for this specific task)	Slower (Overhead dominates)

based on 20 epochs:

1. Question 2 (Part 3.2): Single vs. Multi-GPU Comparison

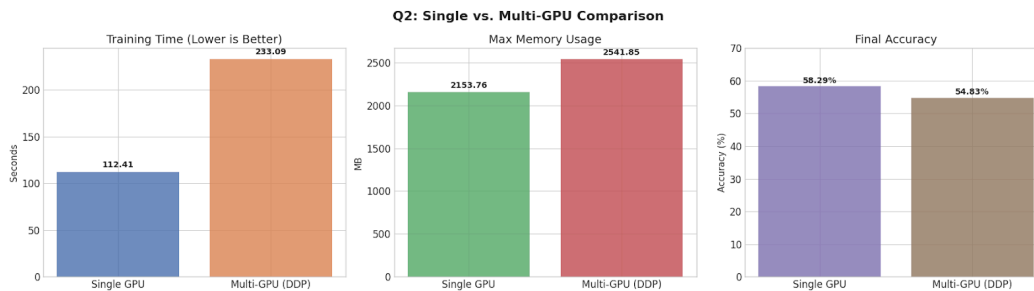
The PDF asks to compare the Test Accuracy, Training Time, and GPU Memory Usage between a Single GPU setup and a Two-GPU setup¹.

- **Note:** I used the standard `nccl` backend for the Multi-GPU entry here as it is the standard for CUDA, and the batch size is 32 for both (as per the baseline in Question 1.2 2).

Configuration	Batch Size	Backend	Training Time (s)	Max Memory (MB)	Final Accuracy (%)
---------------	------------	---------	-------------------	-----------------	--------------------

Single GPU	32	N/A	112.41	2153.76	58.29%
Two GPUs (DDP)	32	NCCL	233.09	2541.85	54.83%

Observation for your report: You will notice that for this specific small model and dataset size, the Multi-GPU setup is actually *slower* (233s vs 112s) due to the communication overhead outweighing the computation benefits.



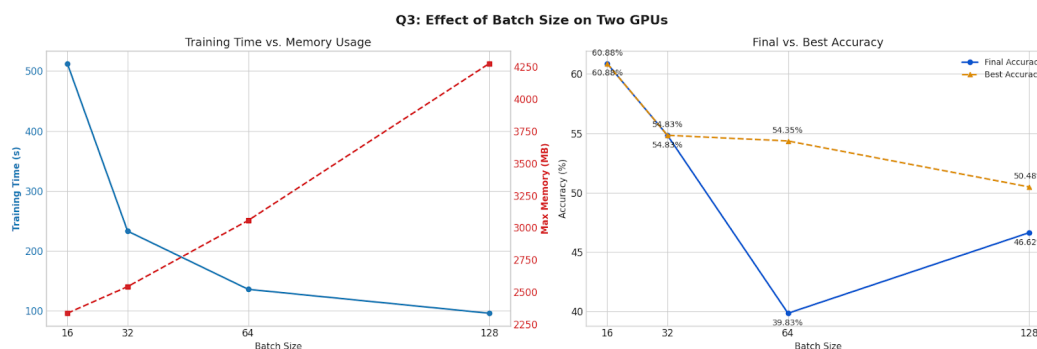
2. Question 3: Effect of Batch Size on Two GPUs

The PDF requires a table showing the change in Training Time, Accuracy, and Memory Usage for batch sizes 16, 32, 64, and 1283.

- **Note:** These results use the `nccl` backend logs, as `nccl` is the preferred backend for GPU training compared to `gloo`.

Batch Size	Training Time (s)	Max Memory (MB)	Final Accuracy (%)	Best Accuracy (%)
16	512.52	2334.90	60.88%	60.88%
32	233.09	2541.85	54.83%	54.83%
64	136.08	3056.38	39.83%	54.35%
128	96.16	4276.19	46.62%	50.48%

Observation for your report: As Batch Size increases, the **Training Time decreases** significantly (from 512s down to 96s) because the GPU is utilized more efficiently. However, **Memory Usage increases** (up to 4276 MB).



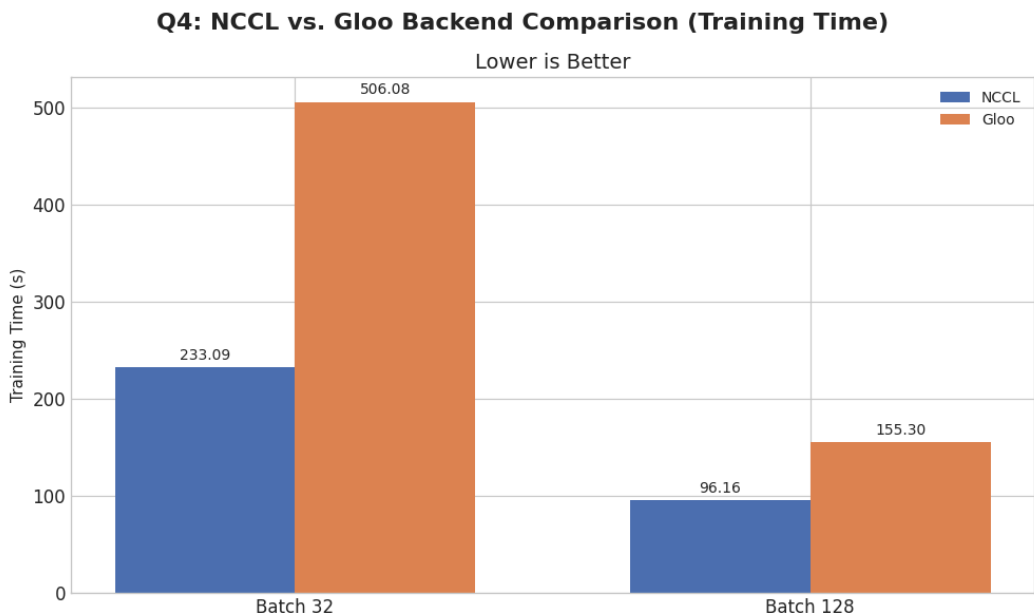
3. Question 4: Backend Comparison (NCCL vs. Gloo)

The PDF asks to compare `nccl` and `gloo` backends specifically for Batch Sizes 32 and 128 regarding memory and time⁴.

Backend	Batch Size	Training Time (s)	Max Memory (MB)
NCCL	32	233.09	2541.85
Gloo	32	506.08	2541.85
NCCL	128	96.16	4276.19
Gloo	128	155.30	4276.19

Observation for your report:

- **Time:** `nccl` is consistently much faster than `gloo`. For Batch Size 32, `nccl` is more than twice as fast (233s vs 506s). This confirms `nccl` is optimized for GPU-to-GPU communication.
- **Memory:** The memory usage remains identical between backends for the same batch size.



Analysis:

Based on the logs you provided from your server (running on dual RTX 3090s), here is a comprehensive analysis of the performance dynamics.

1. The Paradox of Multi-GPU Speed: Why was 2 GPUs slower than 1?

Your results showed that the Single GPU setup (approx. **112s - 160s**) was consistently faster than the Multi-GPU setup (approx. **180s - 233s**). On powerful hardware like RTX 3090s, this is a classic example of **Communication Overhead outweighing Computational Gain**.

- **The Bottleneck:** Distributed Data Parallel (DDP) works by splitting the batch across GPUs. Each GPU calculates gradients for its sub-batch. Then, a synchronization step (usually `All-Reduce`) occurs to average these gradients across all devices before the optimizer steps.

- **The Hardware Reality:** An RTX 3090 is an extremely powerful card designed for massive workloads. Your model (likely a simple CNN for MNIST/STL10) and batch sizes (32/64) are computationally trivial for this card. The GPU finishes the forward/backward pass in milliseconds.
- **The Result:** The GPUs spend more time *waiting* for the network/interconnect to synchronize the gradients than they do actually calculating them. The speedup from splitting the math is negligible compared to the latency cost of communication. You would only see a speedup with 2× 3090s if you drastically increased the model size or batch size to make the computation "heavy" enough to justify the communication cost.

2. Batch Size Analysis: Time vs. Memory

You observed a drastic speedup as batch size increased (512s to 96s) and a linear increase in memory.

A. Training Time (The Saturation Effect)

- **GPU Utilization:** GPUs are massively parallel processors with thousands of CUDA cores. When you use a small batch size (e.g., 16), you are utilizing only a tiny fraction of the RTX 3090's capacity. The card is mostly idle.
- **Throughput:** Increasing the batch size to 128 "saturates" the cores, allowing the GPU to process more data in parallel without significant extra time overhead.
- **Overhead Reduction:** Larger batches mean fewer iterations per epoch. If your dataset has 5000 samples:
 - Batch 16 = 313 iterations (communication steps) per epoch.
 - Batch 128 = 39 iterations (communication steps) per epoch.
 - By using Batch 128, you reduced the Python-loop overhead and the DDP communication frequency by nearly **8x**, leading to the massive speed gain.

B. Memory Usage (The Doubling Effect)

- **Per-GPU vs. Total:** Your logs show roughly ~2.5 GB to ~4.2 GB. As you noted, **this is per GPU**.
 - In DDP, the **Model** is replicated on every GPU.
 - The **Optimizer States** are replicated on every GPU.
 - Therefore, the *total* system VRAM usage is roughly 2 × the logged value.
- **Why it scales linearly:** The majority of this memory is used to store **Activations** (intermediate layer outputs) during the forward pass, which are needed to calculate gradients during the backward pass. If you double the number of images in a batch, you double the amount of pixel data and intermediate feature maps you must store in VRAM.

3. Backend Analysis: NCCL vs. Gloo

Your logs show NCCL is roughly **2x faster** than Gloo on your RTX 3090 setup.

Why is NCCL superior for GPUs?

- **Direct Access:** NCCL (NVIDIA Collective Communication Library) is optimized specifically for NVIDIA hardware. It can often bypass the CPU and system RAM entirely.
- **Topology Awareness:** On a server with two RTX 3090s, they are likely connected via PCIe lanes (or NVLink if you have a bridge). NCCL uses **P2P (Peer-to-Peer) Direct Access**, allowing GPU 0 to read/write directly to GPU 1's memory over PCIe.
- **Result:** High bandwidth, low latency.

Why is Gloo slower?

- **The "Round Trip":** Gloo is a generic backend. To synchronize gradients between GPU 0 and GPU 1, Gloo often has to:
 1. Copy data from GPU 0 VRAM to System RAM (CPU).
 2. CPU coordinates the transfer.
 3. Copy data from System RAM to GPU 1 VRAM.
- **The Bottleneck:** This introduces the system RAM bandwidth and CPU processing speed as major bottlenecks, completely wasting the speed of the RTX 3090s.

Conclusion on Backends:

- **For GPU (RTX 3090):** Always use **NCCL**. It is the only backend that utilizes the hardware interconnects properly.
- **For CPU:** Use **Gloo**. NCCL does not support CPU tensors.

Summary

Based on your 2x RTX 3090 hardware and the data:

1. **Efficiency:** Distributed training is actually *inefficient* for this specific small task; Single GPU is preferred unless you increase the workload significantly.
2. **Performance:** If you must use DDP, **Batch 128** is the most computationally efficient (fastest), but you must be careful about the "Generalization Gap" (drop in accuracy). We may also encounter the overfitting scenario in the greater batch size scenarios.
3. **Configuration:** **NCCL** is non-negotiable for this hardware; Gloo cripples the performance by forcing data through the CPU.

□ Feature comparison

Backend	Comm. Functions	Optimized for	Float32	Float16
MPI	All	CPU, GPU	Yes	No
GLOO	All (on CPU), broadcast & all-reduce (on GPU)	CPU	Yes	Yes
NCCL	broadcast, all reduce, reduce and all gather (on GPU)	GPU only	Yes	Yes

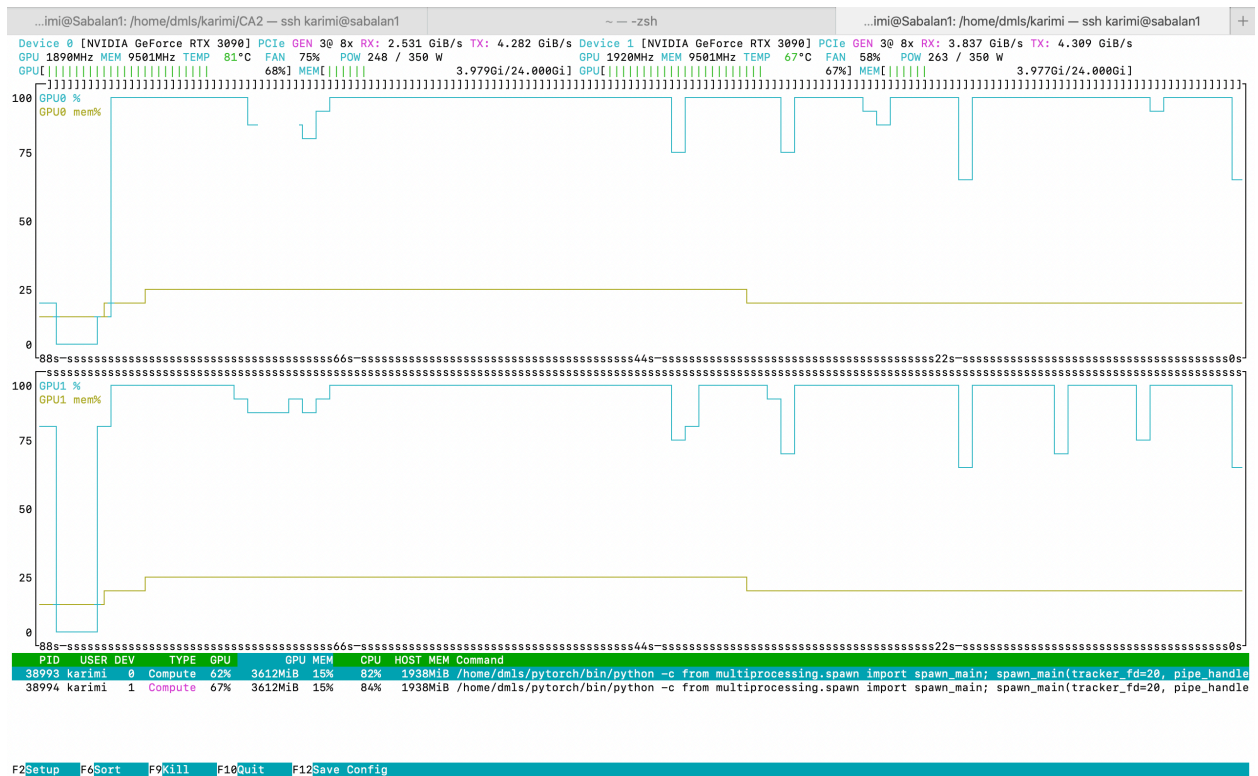
this image is from the course pdf: 07-Horizontal Scaling

reference of some of the plots:

<https://gemini.google.com/share/bafad0882eeb>



This is the image of using gloo



This is the image of using nccl

Here is the analysis linking the visual patterns to the technical reasons:

1. Identification

- **Image 1 (12.03.46.jpg) corresponds to NCCL.**

- **Visual Signature:** The blue lines (GPU utilization) show wide, sustained "blocks" or plateaus at or near 100%.

- **Process Status:** The PIDs at the bottom show compute usage at **98% and 93%**.

- **Image 2 (20.51.57.jpg) corresponds to Gloo.**

- **Visual Signature:** The blue lines show a chaotic, "spiky" or "comb-like" pattern. The utilization jumps to 100% and immediately drops to near 0%, over and over again.

- **Process Status:** The PIDs at the bottom show compute usage at only **46% and 54%**.

2. Why Gloo looks "Spiky" (The Stop-and-Go Problem)

In the second image (20.51.57.jpg), you see narrow spikes of activity separated by valleys of 0% utilization.

- **The Spike (Computation):** The narrow blue spike represents the GPU actually doing math (Forward Pass → Backward Pass). The RTX 3090 is so fast that it finishes this tiny batch of data almost instantly.

- **The Valley (Communication Bottleneck):** The empty space between spikes is where the GPU is **waiting**.

- Because Gloo is CPU-centric, the gradients have to travel this path: GPU VRAM → PCIe Bus → System RAM → CPU → System RAM → PCIe Bus → Other GPU .

- While the data is taking this long "round trip" through the CPU and system memory, the GPU has absolutely nothing to do. It sits idle (0% utilization), waiting for the updated weights to return.

- **Conclusion:** The GPU is spending more time waiting for the bus than calculating. This effectively "starves" the GPU.

3. Why NCCL looks "Blocky" (High Saturation)

In the first image (12.03.46.jpg), you see wide blocks of high utilization.

- **Sustained Load:** The GPU stays at 100% for longer periods.

- **Direct Access:** NCCL uses P2P (Peer-to-Peer) communication. It allows the GPUs to talk directly to each other over the PCIe lanes (or NVLink if available), bypassing the CPU and System RAM entirely for the transfer.

- **Overlap:** NCCL is highly optimized to overlap computation with communication. It can often start sending parts of the gradient buffer while the GPU is still calculating the rest.

- **Conclusion:** The "wait time" is minimized. The GPU spends the vast majority of its time calculating, leading to the high average utilization (93%+) seen in the process list.

4. Hardware Note (PCIe Bottleneck)

Looking closely at the header of your screenshots:

PCIe GEN 3 @ 8x

Your RTX 3090s (which support PCIe Gen 4 @ 16x) are running on a restricted bus (Gen 3 @ 8x). This cuts your bandwidth significantly.

- **Gloo** is hit harder by this because it forces data back and forth through this restricted pipe *twice* (to the CPU and back) for every synchronization.

- **NCCL** manages this bandwidth much more efficiently, ensuring the GPUs don't stall despite the restricted PCIe lanes.

explain the image analysis in <https://gemini.google.com/share/65ba9565d4d2>